

## Python Bisect

- Maintaining a **sorted list** while inserting new elements.
- Quickly finding the **position** where an element should be inserted.
- Implementing **binary search** operations.
- Useful in problems like scheduling, interval management, or ranking systems.

| Method                                                  | Syntax                                    | Description                                                                                                                       | Example                                     |
|---------------------------------------------------------|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| <b>bisect_left</b>                                      | bisect_left(a, x,<br>lo=0,<br>hi=len(a))  | Returns the index where x should be inserted to maintain sorted order, choosing the <b>leftmost</b> position if x already exists. | bisect_left([1,2,4,4,5], 4) → 2             |
| <b>bisect_right</b><br><b>ht</b> (alias <b>bisect</b> ) | bisect_right(a,<br>x, lo=0,<br>hi=len(a)) | Returns the index where x should be inserted, choosing the <b>rightmost</b> position if x already exists.                         | bisect_right([1,2,4,4,5], 4) → 4            |
| <b>insort_left</b>                                      | insort_left(a, x,<br>lo=0,<br>hi=len(a))  | Inserts x into list a at the position found by bisect_left. Keeps list sorted.                                                    | insort_left([1,2,4,5], 4) →<br>[1,2,4,4,5]  |
| <b>insort_right</b><br><b>ht</b> (alias <b>insort</b> ) | insort_right(a,<br>x, lo=0,<br>hi=len(a)) | Inserts x into list a at the position found by bisect_right.                                                                      | insort_right([1,2,4,5], 4) →<br>[1,2,4,4,5] |



## Explanation & Use Cases

### 1. `bisect_left`

- Finds the insertion point **before** existing entries.
- Useful when you want to insert duplicates at the start of their block.
- Example: Ranking systems where ties should appear earlier.

Python ^

```
import bisect
lst = [10, 20, 30, 40]
print(bisect.bisect_left(lst, 30)) # 2
```

### 2. `bisect_right` / `bisect`

- Finds the insertion point **after** existing entries.
- Useful when you want duplicates to appear at the end of their block.
- Example: Scheduling tasks where later duplicates should be placed last.

Python ^

```
lst = [10, 20, 30, 40]
print(bisect.bisect_right(lst, 30)) # 3
```

---

### 3. `insort_left`

- Actually inserts the element at the position given by `bisect_left` .
- Keeps the list sorted automatically.
- Example: Maintaining a sorted leaderboard.

Python ^

```
lst = [10, 20, 40]
bisect.insort_left(lst, 30)
print(lst) # [10, 20, 30, 40]
```

### 4. `insort_right` / `insort`

- Inserts the element at the position given by `bisect_right` .
- Example: Adding events to a timeline where duplicates should appear later.

Python ^

```
lst = [10, 20, 40]
bisect.insort_right(lst, 30)
print(lst) # [10, 20, 30, 40]
```